

Aula 2 — Playwright Avançado

"Você já sabe o que testar. Agora vamos automatizar."

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 01/10/2026

Recap da Aula 1

- ISO/IEC 25010:2023 (9 características), Pirâmide vs Trophy vs Honeycomb
- Teste exploratório: FEW HICCUPPS + Charter (SBTM)
- Web/mobile por dentro (render pipeline, lifecycle), acessibilidade, heurísticas de Nielsen
- Lab manual: app TestesQAMobile (13+ bugs) + exploratório no arngren.net

Atividade 1 entregue? (Módulos 1, 2, 3 do app + arngren.net — prazo era hoje)

| Hoje muda de fase: tudo que vocês testaram na unha, começamos a automatizar.

Agenda da aula (3h30)

- **90min** — Playwright avançado: arquitetura interna, locators, network mocking, visual regression (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: Playwright SPA (live coding com replicação pelos alunos)
- **15min** — Wrap-up

Objetivos M2

- **Compreender** arquitetura interna do Playwright (CDP, contexts)
- **Aplicar** locators recomendados (role, text, test-id)
- **Implementar** network mocking + HAR replay
- **Configurar** storageState para auth reuse
- **Aplicar** visual regression nativo (`toHaveScreenshot`)
- **Configurar** sharding 4-way em CI

O que é o Playwright?

Framework de automação de browser — controla Chromium, Firefox e WebKit de fora, como um robô clicando e digitando, mas via protocolo, não coordenadas de tela.

Origem: criado pela **Microsoft**, lançado janeiro/2020. Time = os mesmos engenheiros que criaram o **Puppeteer** no Google, saíram e foram pra Microsoft em 2019 — usaram a experiência pra corrigir as limitações do Puppeteer (só Chromium).

3 coisas que o Puppeteer nunca teve: suporte a Firefox e WebKit (não só Chrome) · API multi-página pensada pra SPA · auto-waiting nativo (sem `waitForSelector` espalhado pelo código).

Fonte: [Wikipedia, Playwright \(software\)](#)

Instalando e gerando testes sozinho

```
# Cria projeto novo com tudo configurado (TS, config, exemplo)
npm init playwright@latest

# Baixa os browsers (Chromium, Firefox, WebKit) — ~300MB
npx playwright install
```

Codegen — grava sua navegação e gera o teste sozinho:

```
npx playwright codegen https://demo.playwright.dev/todomvc
```

Abre um browser real. Você clica, digita, navega normalmente — o Playwright **grava cada ação** e escreve o código do teste ao vivo numa janela ao lado (locators já na forma recomendada). Ótimo pra começar sem saber a API de cor.

| Rodem isso agora, ao vivo, em qualquer site — depois a gente edita o que ele gerou.

Por que Playwright?

Em 2026, Playwright tomou espaço de Selenium e Cypress em times sérios.

Razões técnicas:

1. **Chrome DevTools Protocol** (não WebDriver) — controle granular
2. **Browser contexts isolados** — paralelismo limpo
3. **Auto-waiting nativo** — sem sleeps
4. **Locators recomendados** — role, text, test-id
5. **Trace viewer** — debug determinístico
6. **Cross-browser** — Chromium, Firefox, WebKit
7. **Visual regression nativo** — `toHaveScreenshot`

Arquitetura — CDP vs WebDriver

Selenium (WebDriver):

- Protocolo HTTP REST padronizado W3C
- Compatível com infinitos browsers
- Cada comando = chamada HTTP

Playwright (CDP):

- Protocolo de baixo nível (Chrome DevTools)
- Conexão WebSocket persistente
- Acesso granular a network, runtime, performance

Diferença prática: Playwright é tipicamente 2–4x mais rápido em mesma suíte.

Browser context — isolamento

```
const browser = await chromium.launch();

// Cada context é isolado: cookies, storage, history
const ctx1 = await browser.newContext({ locale: 'pt-BR' });
const ctx2 = await browser.newContext({ locale: 'en-US' });

const page1 = await ctx1.newPage();
const page2 = await ctx2.newPage();
```

| 1 browser, N contexts, M pages — paralelismo natural.

Locators recomendados

```
// ✅ Bom — role + nome
page.getByRole('button', { name: 'Entrar' });
page.getByRole('link', { name: 'Sobre nós' });

// ✅ Bom — texto visível
page.getByText('Welcome back');

// ✅ Bom — test-id (estável)
page.getByTestId('submit-button');

// ✅ Bom — placeholder, label
page.getByPlaceholder('Email');
page.getByLabel('Senha');

// ❌ Ruim — frágil
page.locator('div.container > .card:nth-child(2) span.title');
```

Auto-waiting — sem sleeps

```
// Playwright aguarda elemento estar:  
// - Anexado ao DOM  
// - Visível  
// - Estável (não animando)  
// - Habilitado  
// - Receivable (não coberto)  
  
await page.getByRole('button', { name: 'Entrar' }).click();  
// Já aguardou tudo automaticamente  
  
// Web-first assertions também aguardam  
await expect(page.getByText('Welcome')).toBeVisible({ timeout: 5000 });
```

Se ainda precisa de `waitForTimeout`, está fazendo errado.

Flakiness — métricas

Teste flaky: mesmo código, mesma branch, comportamento diferente entre runs.

```
Flake rate = runs flaky / total runs
```

Targets:

- Time elite: < 1% flake rate
- Time saudável: < 5%
- Time problemático: > 10%

```
# Detectar flaky em CI  
npx playwright test --repeat-each=3  
# Testes que falharam em 1 ou 2 (não 0 nem 3) são flaky
```

Flakiness — mitigação

- ✓ **Auto-waiting** (Playwright nativo — slide anterior)
- ✓ **Locators recomendados** (role, text, test-id) > seletores frágeis
- ✓ **Isolamento entre testes** (storageState, browser context)
- ✓ **Retry inteligente** em CI (1–2 max, com investigação)
- ✗ **Sleep fixo** (`waitForTimeout`) — anti-pattern absoluto
- ✗ **Retry em massa** (`retries: 10`) — mascara bugs reais
- ✗ **Skip silencioso** — esconde problemas

Regra prática: cada flake tem causa raiz. "Rodei de novo e passou" não é debug, é procrastinação.

Network mocking — `route()`

```
test('handles API error gracefully', async ({ page }) => {  
  // Intercepta antes da request  
  await page.route('**/api/checkout', async (route) => {  
    await route.fulfill({  
      status: 500,  
      contentType: 'application/json',  
      body: JSON.stringify({ error: 'Internal Server Error' }),  
    });  
  });  
  
  await page.goto('/checkout');  
  await page.getByRole('button', { name: 'Pagar' }).click();  
  
  await expect(page.getByText('Erro. Tente novamente.')).toBeVisible();  
});
```

HAR replay

Captura tráfego real e replay determinístico em testes.

```
// Capturar
const context = await browser.newContext({
  recordHar: { path: 'session.har' },
});

// Replay
await page.routeFromHAR('session.har', {
  url: '**/api/**',
  notFound: 'fallback',
});
```

Útil para testar fluxos complexos sem depender de backend rodando.

storageState — auth reuse

Sem reuso: cada teste faz login (3s × 200 testes = 10min só de login).

```
// global-setup.ts — roda 1 vez
async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('/login');
  await page.fill('[name=email]', 'user@email.com');
  await page.fill('[name=password]', 'secret');
  await page.click('button[type=submit]');
  await page.waitForURL('/dashboard');

  await page.context().storageState({ path: 'auth.json' });
  await browser.close();
}
```

```
// playwright.config.ts
export default defineConfig({
  globalSetup: require.resolve('./global-setup'),
  use: { storageState: 'auth.json' },
});
```

Visual regression nativo

```
test('home page visual', async ({ page }) => {  
  await page.goto('/');  
  await expect(page).toHaveScreenshot('home.png', {  
    maxDiffPixels: 100,  
  });  
});  
  
// Tirar baseline na primeira execução:  
// npx playwright test --update-snapshots
```

Para múltiplos viewports, configure projects no `playwright.config.ts`.

Trace viewer — debug determinístico

```
// playwright.config.ts
use: {
  trace: 'retain-on-failure', // captura trace em failures
}
```

```
# Abrir trace de execução
npx playwright show-trace trace.zip
```

UI mostra:

- Timeline de cada ação
- DOM snapshots a cada step
- Network requests
- Console logs
- Source code do teste

Component Testing

Alternativa a Cypress CT.

```
// LoginForm.spec.tsx
import { test, expect } from '@playwright/experimental-ct-react';
import LoginForm from './LoginForm';

test('disables submit when email invalid', async ({ mount }) => {
  const component = await mount(<LoginForm />);
  await component.locator('[name=email]').fill('invalid');
  await expect(component.locator('button[type=submit]')).toBeDisabled();
});
```

Roda componente isolado em browser real, sem app inteiro. Tempo médio entre unit e E2E.

Page Object Model (POM)

Padrão clássico. Encapsular interação com página em uma classe — o teste não conhece seletores, só a intenção.

```
class LoginPage {  
    constructor(private page: Page) {}  
    emailInput = this.page.getByPlaceholder('Email');  
    submitButton = this.page.getByRole('button', { name: 'Entrar' });  
  
    async login(email: string, password: string) {  
        await this.emailInput.fill(email);  
        await this.page.getByPlaceholder('Senha').fill(password);  
        await this.submitButton.click();  
    }  
}  
  
test('login flow', async ({ page }) => {  
    await new LoginPage(page).login('user@email.com', 'secret');  
    await expect(page).toHaveURL('/dashboard');  
});
```

POM — por que (e quando não)

Por que: seletor mudou? Corrige em 1 lugar (a classe), não em 50 testes. Teste lê como intenção de negócio, não como manipulação de DOM.

Fixtures do Playwright são a evolução natural: injetam a Page Object pronta via `test.extend()`, sem `new LoginPage(page)` repetido em todo teste.

```
export const test = base.extend<{ loginPage: LoginPage }>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
});
```

Critério da Atividade aceita **POM clássico ou Fixtures** — Fixtures é mais idiomático em Playwright 2026, mas os dois pontuam igual.

Screenplay Pattern — pra saber que existe

Evolução do POM: modela **Actor**, **Ability**, **Task**, **Question** em vez de classes por página.

```
const user = Actor.named('Maria').whoCan(BrowseTheWeb.using(page));  
await user.attemptsTo(Login.withCredentials('user@email.com', 'secret'));  
await user.asks(TheTitle.ofPage()).should(equal('Dashboard'));
```

Mais verboso, mais expressivo — vale a pena em suítes grandes com muito reuso entre fluxos. **Não vamos aprofundar hoje** (POM já resolve 90% dos casos reais); se quiser se aprofundar: [Serenity/JS](#), [The Screenplay Pattern](#).

Sharding 4-way em CI

```
# .github/workflows/playwright.yml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      shard: [1/4, 2/4, 3/4, 4/4]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shard }}
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: blob-report-${{ matrix.shard }}
          path: blob-report
```

Blob reports — agregação

```
merge-reports:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
    - run: npx playwright merge-reports --reporter html ./blob-report
    - uses: actions/upload-artifact@v4
      with: { name: html-report, path: playwright-report }
```

Resultado: 1 relatório consolidado de 4 shards paralelos.

Lab — parte 1: esquentando guiado (30min)

Fazemos juntos, projetado, todo mundo replica — app **TodoMVC** (demo oficial do Playwright, sempre no ar).

1. Clone `exercicios/00-warmup-aula1-playwright` (5min)
2. Criar 3 testes E2E básicos (locators + auto-waiting) (10min)
3. Aplicar Page Object Model na suíte (10min)
4. Rodar com trace habilitado, abrir o trace viewer (5min)

Roteiro completo: `ROTEIRO0-AULA1.md` na pasta do exercício. Não vale nota — é prática antes do lab de verdade.

Lab — parte 2: Atividade real (60min)

Agora no exercício **avaliativo** — `exercicios/01-lab-web-pwa-playwright/` (CineFav Web, 20pts, Aula 2+3).

1. Fork + clone do repo da disciplina (5min)
2. `cd pratica/` → `npm install` → `npm run test:e2e` (roda "vazio") (5min)
3. Spec **02 — network mocking**: completar os TODOs (20min)
4. Spec **03 — visual regression**: baseline em 3 viewports (20min)
5. Auth reuse (`storageState`) já usado no `auth.setup.ts` — revisar juntos (10min)

Specs 04 (SW lifecycle) e 05 (offline) ficam pra Aula 3 — dependem de conteúdo que ainda não vimos.

Pitfalls comuns

- ✗ `waitForTimeout` em vez de auto-waiting
- ✗ Locators frágeis dependentes de hierarquia DOM
- ✗ Mock de tudo — testes desacoplam de realidade
- ✗ `retries: 3` sem investigar root cause de flake
- ✗ CI sem sharding — feedback de horas em suítes grandes
- ✗ Sem trace habilitado — debug demanda reproduzir local

Atividade 1 (Lab Web+PWA) — 20 pts, Aula 2 + Aula 3

Entrega: fork + Pull Request no repo da disciplina. App **CineFav Web** já pronto — vocês só escrevem os testes.

#	Critério	Pontos	Quando
1	Auth state reuse (<code>storageState</code>)	3	Hoje
2	Network mocking (<code>route()</code>)	3	Hoje
3	Visual regression, 3 viewports	4	Hoje
4	SW lifecycle testado	3	Aula 3
5	Offline mode (<code>setOffline</code>)	3	Aula 3
6	Lighthouse CI, 3 budgets	4	Aula 3

Critério eliminatório: suíte passa 100% em 3 runs consecutivos — flakiness é bug do teste, não azar. Enunciado completo: `exercicios/01-lab-web-pwa-playwright/enunciado.md`.

Leituras obrigatórias

- [Playwright Docs — Best Practices](#) (oficial)
- Winteringham, M. — *Testing Web APIs* (Manning), cap 5–7
- Axelrod, A. — *Complete Guide to Test Automation* (Apress), cap 8–10
- [Google Testing Blog — Just Say No to More End-to-End Tests](#)
- [Serenity/JS — The Screenplay Pattern](#) (opcional, ver slide de Screenplay)

Próximo módulo (M3)

Testes de SPA & PWA

- Hydration testing
- Service Worker testing
- Offline mode tests
- Background Sync verification
- Lighthouse CI com budgets

Continuação da mesma Atividade 1 (critérios 4-6) — não é um exercício novo.

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Office hours: quintas 19h